



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Kgs. Muhammad Alialjan Nama
NPM	2428240140
Kelas	SI5B
Topik Tugas 1	Topik 33 — Toko Tani: Pupuk
Entitas utama	Pupuk (fertilizer)
Tautan repositori	https://github.com/alialjan/si5B_kuis1_alialjan
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Topik 33 — Toko Tani: Pupuk
Entitas utama (tunggal)	Pupuk (fertilizer)
Alamat dasar	/
Field data	id, namaPupuk, jenis, beratKg, harga, stok
Field wajib saat POST	namaPupuk, jenis (organik/anorganik), beratKg, harga; stok opsional (default 0)
Query string untuk filter	jenis — GET /fertilizers?jenis=organik
Data awal	4 data: Pupuk Kompos Super, Pupuk Urea Granul, Pupuk Kandang Sapi, Pupuk NPK Mutiara
Modul lain (jika ada)	Tidak ada (hanya GET / untuk informasi API)

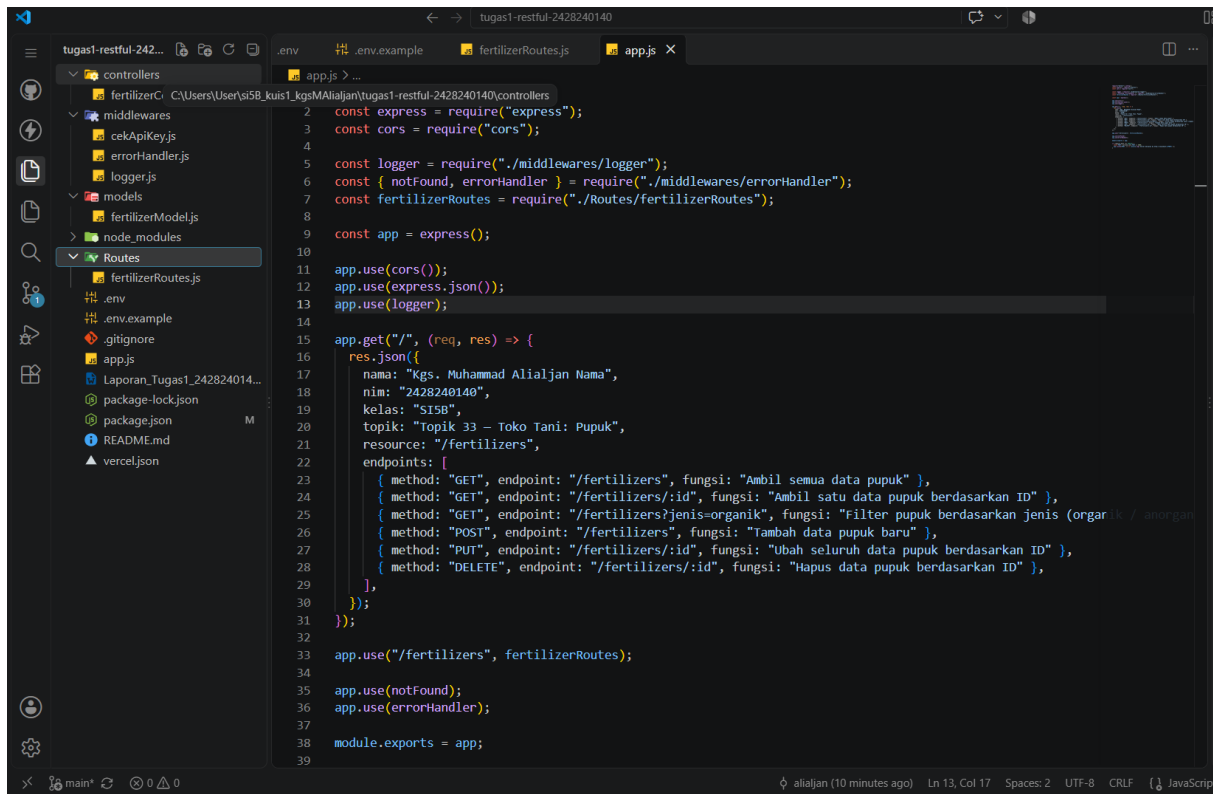
Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/fertilizers	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/fertilizers/:id	Tidak	200	Ambil satu data
3	POST	/fertilizers	Ya	201	Tambah data
4	PUT	/fertilizers/:id	Ya	200	Ubah data
5	DELETE	/fertilizers/:id	Ya	200	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Inst



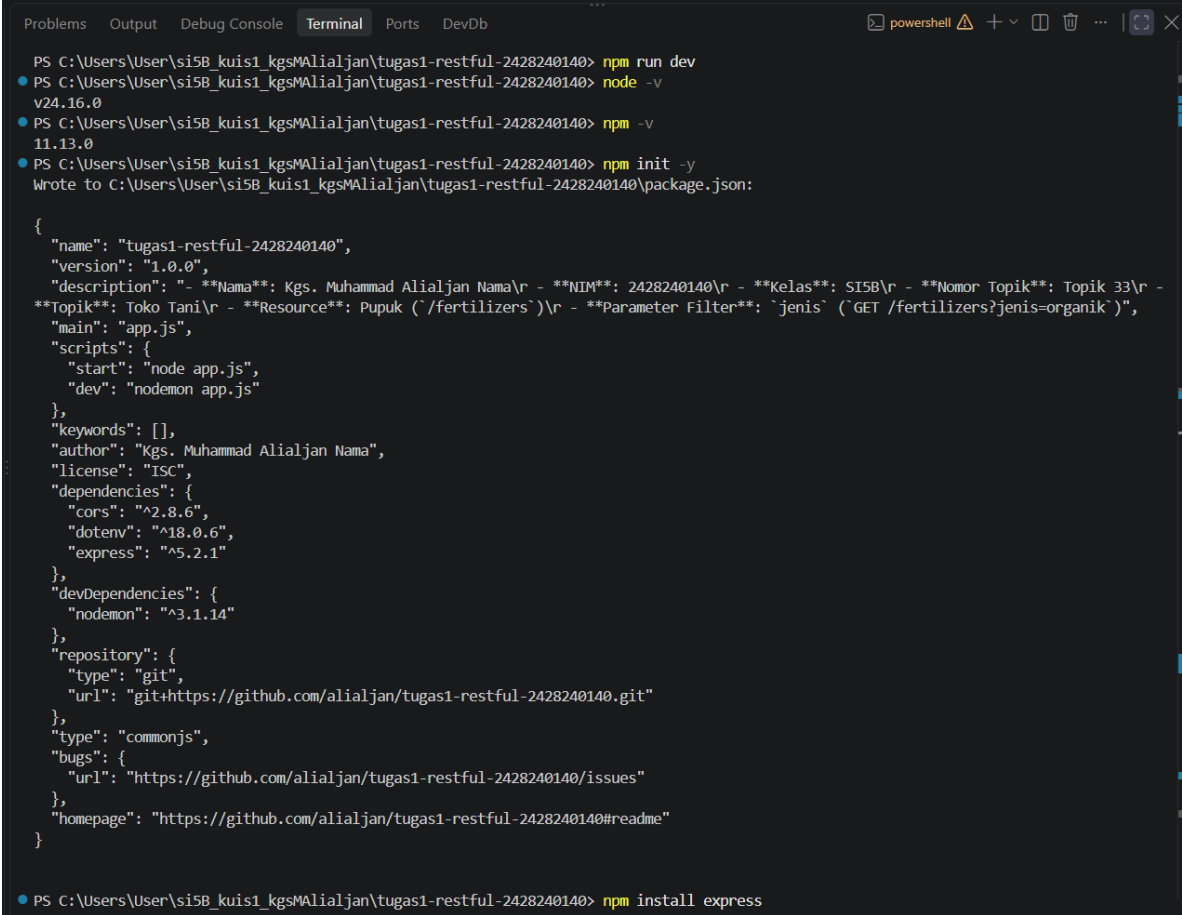
```
1 const express = require("express");
2 const cors = require("cors");
3
4 const logger = require("../middlewares/logger");
5 const { notFound, errorHandler } = require("../middlewares/errorHandler");
6 const fertilizerRoutes = require("../Routes/fertilizerRoutes");
7
8 const app = express();
9
10 app.use(cors());
11 app.use(express.json());
12 app.use(logger);
13
14 app.get("/", (req, res) => {
15   res.json({
16     nama: "Kgs. Muhammad Alialjan Nama",
17     nim: "2428240140",
18     kelas: "SI5B",
19     topik: "Topik 33 – Toko Tani: Pupuk",
20     resource: "/fertilizers",
21     endpoints: [
22       { method: "GET", endpoint: "/fertilizers", fungsi: "Ambil semua data pupuk" },
23       { method: "GET", endpoint: "/fertilizers/:id", fungsi: "Ambil satu data pupuk berdasarkan ID" },
24       { method: "GET", endpoint: "/fertilizers?jenis=organik", fungsi: "Filter pupuk berdasarkan jenis (organik / anorganik)" },
25       { method: "POST", endpoint: "/fertilizers", fungsi: "Tambah data pupuk baru" },
26       { method: "PUT", endpoint: "/fertilizers/:id", fungsi: "Ubah seluruh data pupuk berdasarkan ID" },
27       { method: "DELETE", endpoint: "/fertilizers/:id", fungsi: "Hapus data pupuk berdasarkan ID" },
28     ],
29   });
30 });
31
32 app.use("/fertilizers", fertilizerRoutes);
33
34 app.use(notFound);
35 app.use(errorHandler);
36
37 module.exports = app;
```

alasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



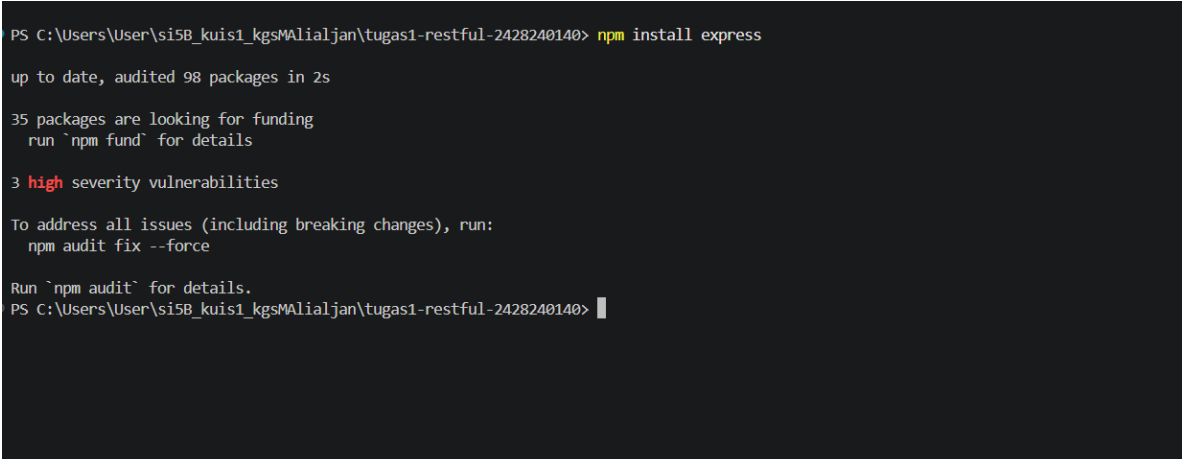
```

PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> npm run dev
● PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> node -v
v24.16.0
● PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> npm -v
11.13.0
● PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> npm init -y
Wrote to C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140\package.json:

{
  "name": "tugas1-restful-2428240140",
  "version": "1.0.0",
  "description": "- **Nama**: Kgs. Muhammad Alialjan Nama\r - **NIM**: 2428240140\r - **Kelas**: SI5B\r - **Nomor Topik**: Topik 33\r - **Topik**: Toko Tani\r - **Resource**: Pupuk (/fertilizers)\r - **Parameter Filter**: `jenis` (`GET /fertilizers?jenis=organik`)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "keywords": [],
  "author": "Kgs. Muhammad Alialjan Nama",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.6",
    "express": "^5.2.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/alialjan/tugas1-restful-2428240140.git"
  },
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/alialjan/tugas1-restful-2428240140/issues"
  },
  "homepage": "https://github.com/alialjan/tugas1-restful-2428240140#readme"
}

● PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> npm install express

```



```

PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140> npm install express

up to date, audited 98 packages in 2s

35 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\User\si5B_kuis1_kgsMALialjan\tugas1-restful-2428240140>

```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi express, cors, dan dotenv, serta mengapa versi Node.js perlu memenuhi syarat Express 5.

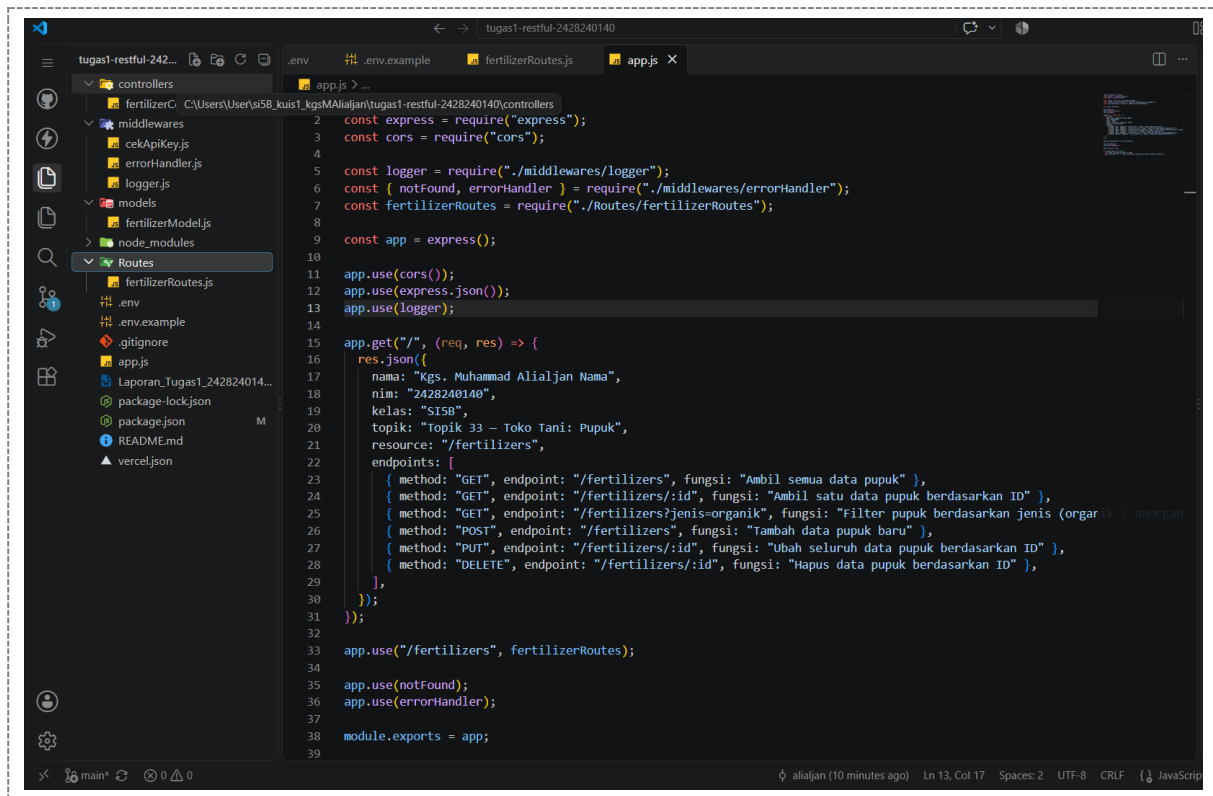
express saya pakai untuk membuat server dan mengatur routing (`express.Router()`); cors dipasang lewat `app.use(cors())` agar API bisa diakses dari asal lain seperti aplikasi front-end; dotenv membaca berkas `.env` ke `process.env` sehingga PORT dan API_KEY tidak tertulis di dalam kode. Express 5 membutuhkan Node.js versi baru, dan Node.js saya v24.16.0 sehingga syaratnya terpenuhi.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

models menyimpan array fertilizers dan fungsi pengolahannya; controllers berisi validasi dan pengiriman response; routes memetakan alamat /fertilizers ke fungsi controller; middlewares berisi logger, cekApiKey, dan errorHandler yang berjalan di antara request dan response.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

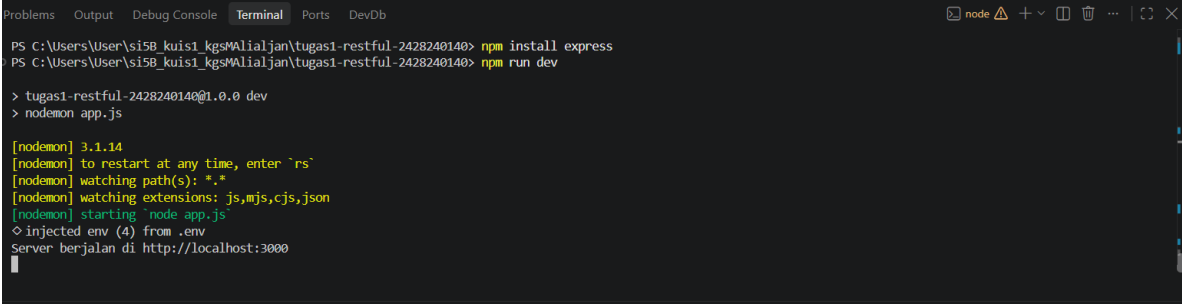
Berkas	Penjelasan Anda
models/fertilizerModel.js	Menyimpan array fertilizers dan nextId, serta fungsi getAll, getById, create, update, dan remove. Berkas ini tidak memakai req dan res, hanya mengolah data.

Berkas	Penjelasan Anda
controllers/fertilizerController.js	Berisi getAllFertilizers, getFertilizerById, createFertilizer, updateFertilizer, dan deleteFertilizer. Fungsi validasi() memeriksa body, lalu controller memanggil model dan mengirim response (200, 201, 400, 404).
routes/fertilizerRoutes.js	Membuat express.Router() yang memetakan GET, POST, PUT, DELETE ke fungsi controller. cekApiKey dipasang pada rute POST, PUT, dan DELETE saja.
middlewares/logger.js	Mencetak waktu, metode, dan alamat setiap request ke terminal, lalu memanggil next().
middlewares/cekApiKey.js	Membandingkan header x-api-key dengan process.env.API_KEY. Jika tidak cocok, langsung membalas 401 dan next() tidak dipanggil.
middlewares/errorHandler.js	notFound membalas 404 untuk alamat yang tidak terdaftar. errorHandler menangkap JSON rusak (entity.parse.failed) menjadi 400 dan error lain menjadi 500, semuanya berformat JSON.
app.js	Memuat dotenv, memasang cors, express.json(), dan logger, membuat rute GET / untuk info API, memasang router /fertilizers, notFound, errorHandler, lalu menjalankan app.listen pada PORT.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
Problems Output Debug Console Terminal Ports DevDb
PS C:\Users\User\si5B_kuis1_kgsMAlialJan\tugas1-restful-2428240140> npm install express
PS C:\Users\User\si5B_kuis1_kgsMAlialJan\tugas1-restful-2428240140> npm run dev

> tugas1-restful-2428240140@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:3000
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Nomor port diambil dari `process.env.PORT` yang dibaca dari berkas `.env` oleh `dotenv`; jika tidak ada, dipakai nilai bawaan 3000 pada `app.js`. Jika port sudah dipakai aplikasi lain, Node.js menampilkan error `EADDRINUSE` dan server gagal berjalan, sehingga `PORT` di `.env` harus diganti atau aplikasi lain dihentikan.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

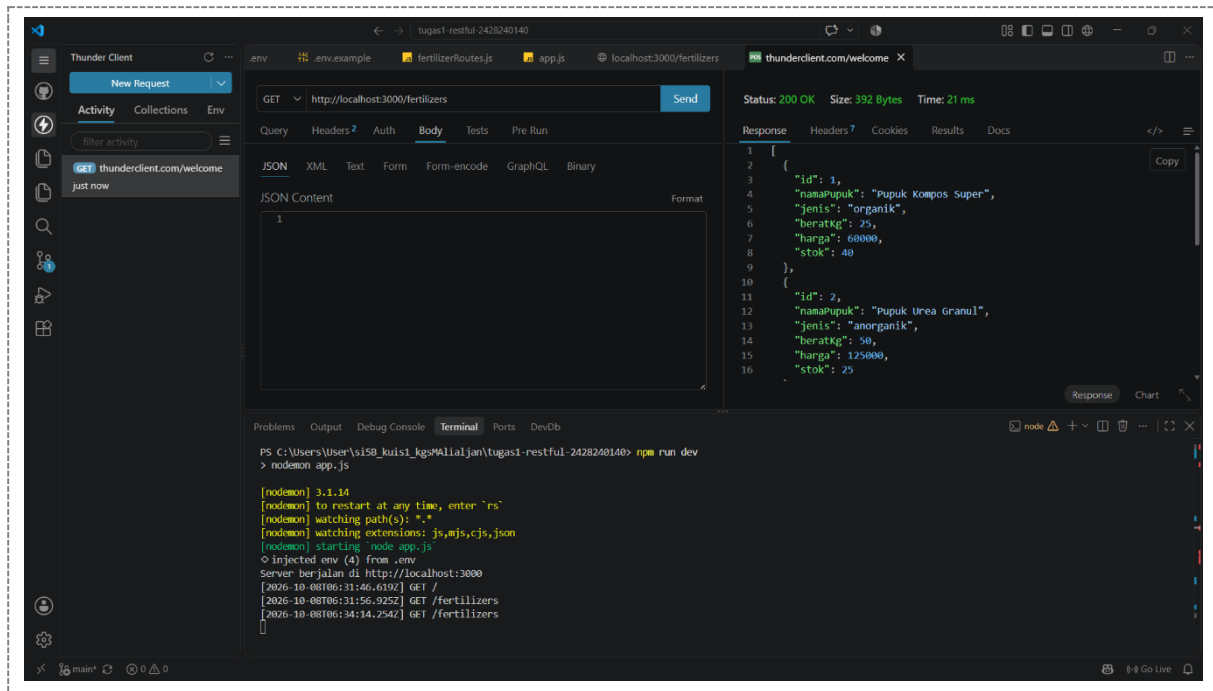
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /fertilizers)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /fertilizers
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK — larik JSON berisi 4 data pupuk (id 1 sampai 4)
Lulus?	☐ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

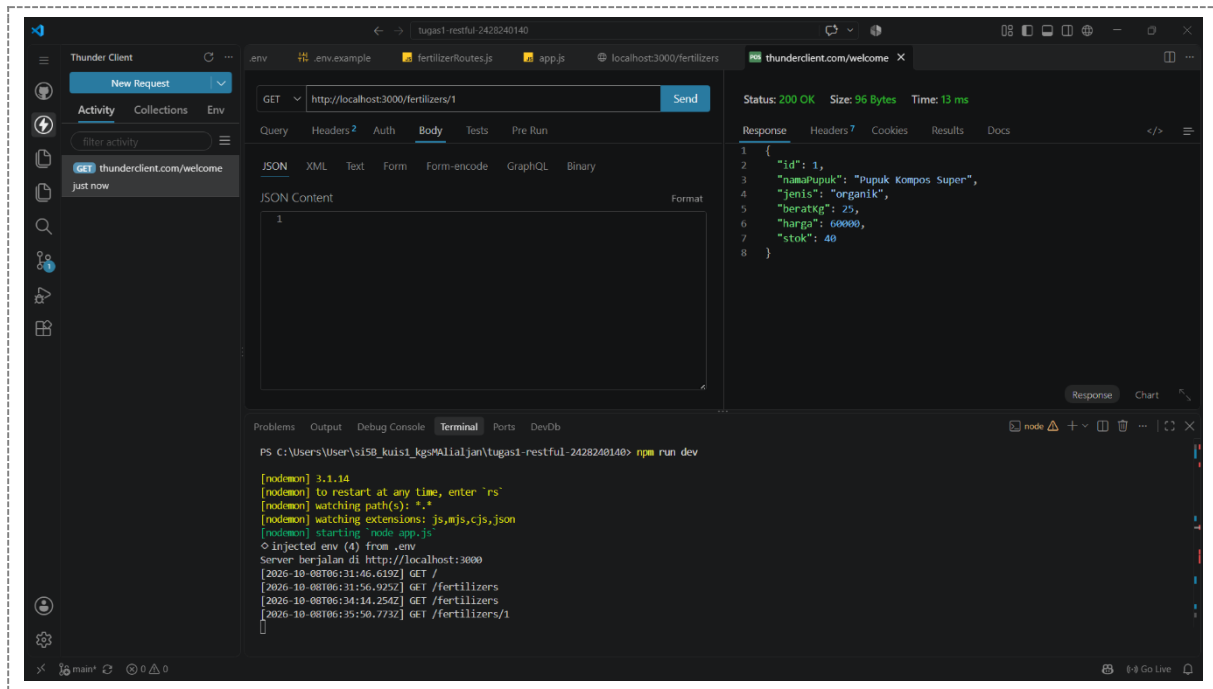
Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Kode 200 berarti permintaan berhasil, dan endpoint ini mengembalikan seluruh data pupuk (atau data terfilter bila memakai `?jenis=`). Alurnya: rute `router.get("/")` di `fertilizerRoutes.js` memanggil `getAllFertilizers` di controller, yang memanggil `Fertilizer.getAll()` di model.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /fertilizers/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK — objek pupuk id 1 (Pupuk Kompos Super)
Lulus?	☐ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

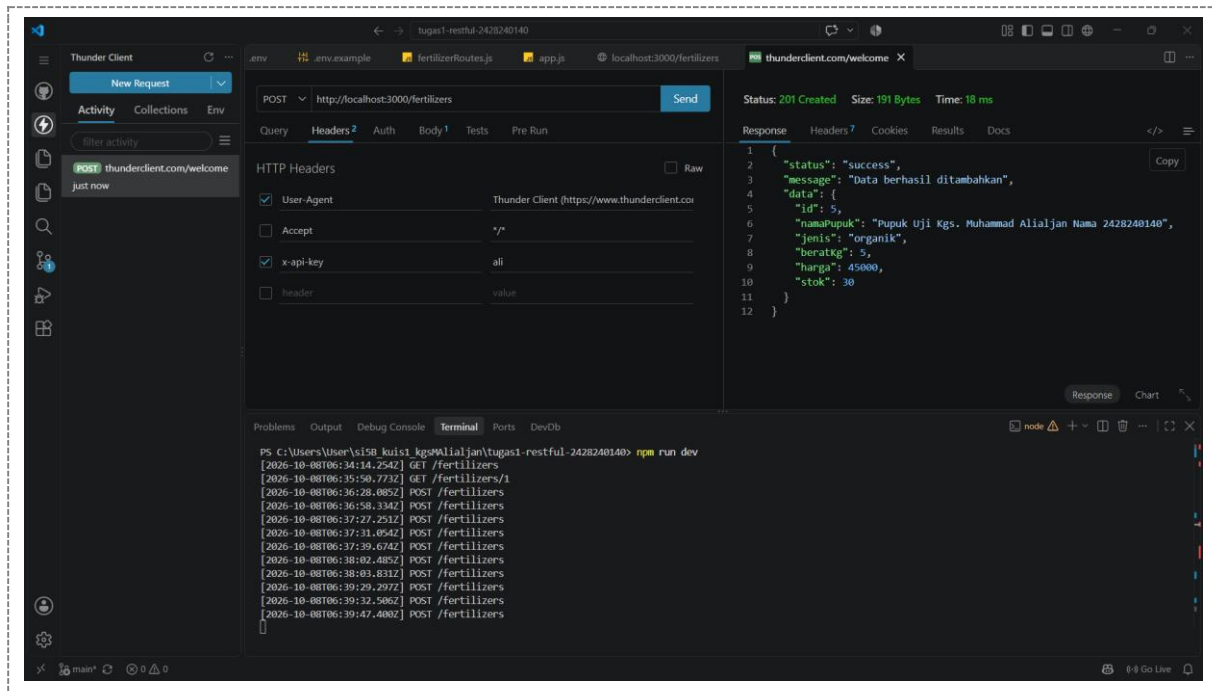
Jelaskan bagaimana ID dari alamat diambil (`req.params`) dan mengapa perlu diubah menjadi angka.

ID diambil dari alamat lewat `req.params.id`, yang berupa string. Nilainya diubah dengan `parseInt` agar sama tipenya dengan `id` bertipe angka pada model, karena `getById` memakai perbandingan ketat (`===`).

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /fertilizers
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPupuk": "Pupuk Uji Kgs. Muhammad Alialjan Nama 2428240140", "jenis": "organik", "beratKg": 5, "harga": 45000, "stok": 30 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created — "Data berhasil ditambahkan", id baru 5
Lulus?	[] Ya [] Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

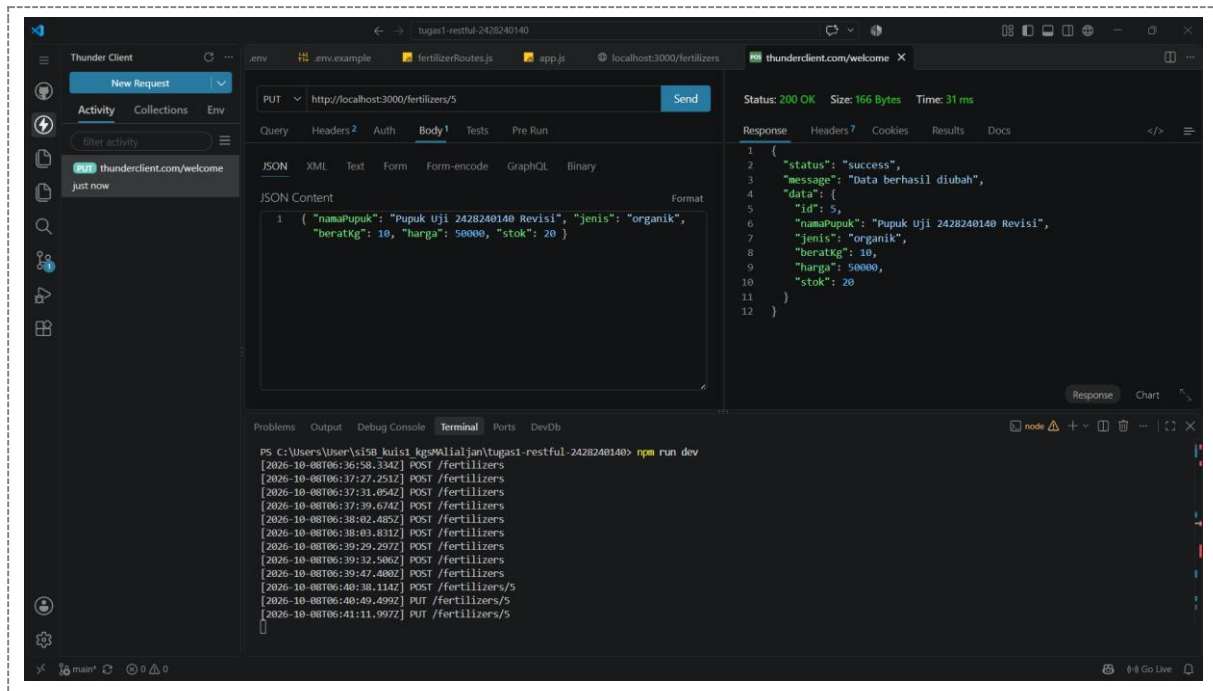
Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

Header `x-api-key` diperiksa oleh middleware `cekApiKey` yang dipasang pada rute POST, lalu dibandingkan dengan `API_KEY` di `.env`. `express.json()` di `app.js` mengubah body JSON menjadi `req.body` agar bisa divalidasi controller. Status 201 dipakai karena ada data baru yang dibuat (id 5), bukan sekadar permintaan berhasil seperti 200.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /fertilizers/5
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPupuk": "Pupuk Uji 2428240140 Revisi", "jenis": "organik", "beratKg": 10, "harga": 50000, "stok": 20 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK — "Data berhasil diubah", data id 5 berubah
Lulus?	[] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

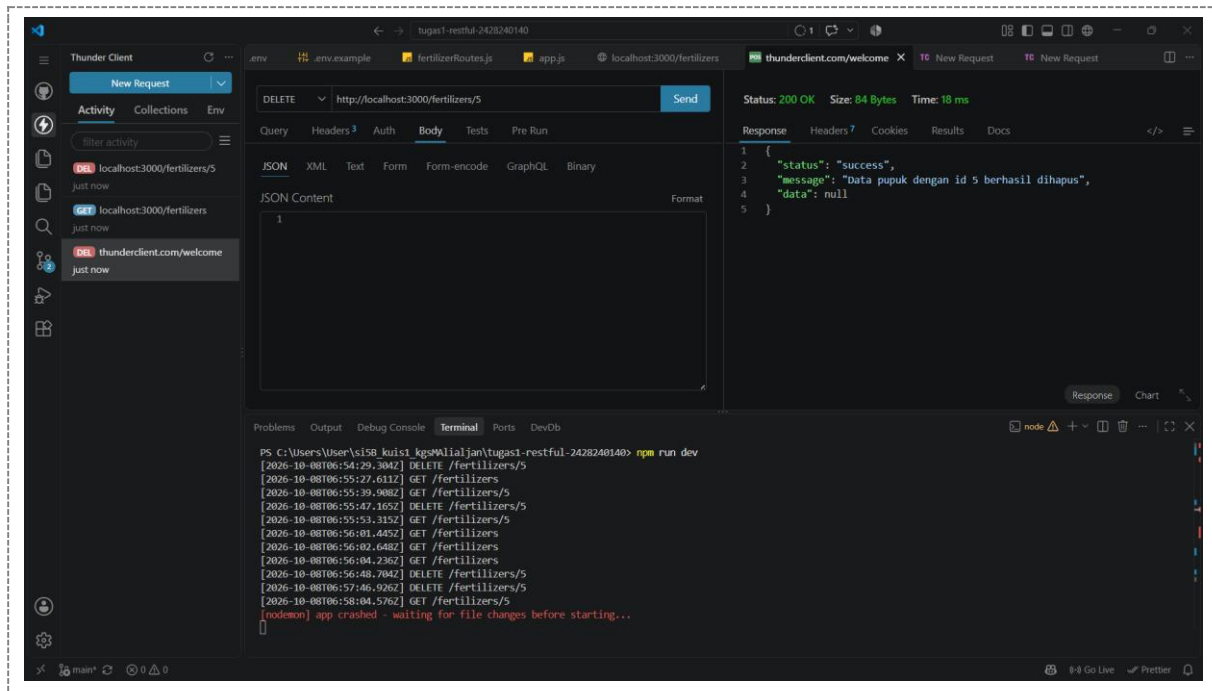
Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Field namaPupuk, beratKg, harga, dan stok berubah, sedangkan id 5 dan jenis "organik" tetap. Perubahannya dilakukan fungsi update() di model yang mengganti seluruh objek, sedangkan controller hanya memeriksa id, memvalidasi body, dan mengirim response 200.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /fertilizers/5
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; JSON berisi status success dan data null (templat menulis 204, API saya mempertahankan perilaku Tugas 1).
Status hasil uji	200 OK — "Data pupuk dengan id 5 berhasil dihapus", data: null
Lulus?	☐ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Status 204 berarti berhasil tanpa isi respons. Pada API saya yang muncul 200 OK dengan JSON status success dan data null, karena refactor harus mempertahankan perilaku Tugas 1; penghapusan dilakukan `remove()` di model dan pesannya dikirim `deleteFertilizer` di controller.

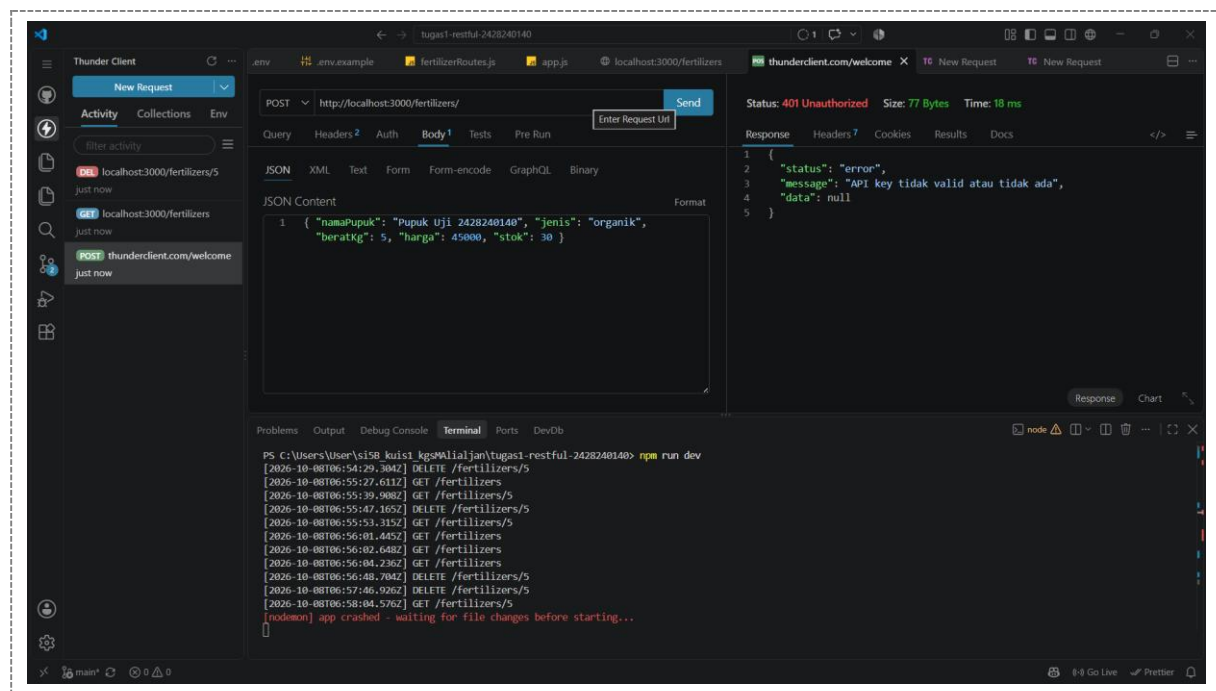
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /fertilizers (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "namaPupuk": "Pupuk Uji 2428240140", "jenis": "organik", "beratKg": 5, "harga": 45000, "stok": 30 }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized — "API key tidak valid atau tidak ada"
Lulus?	[] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

Penjelasan:

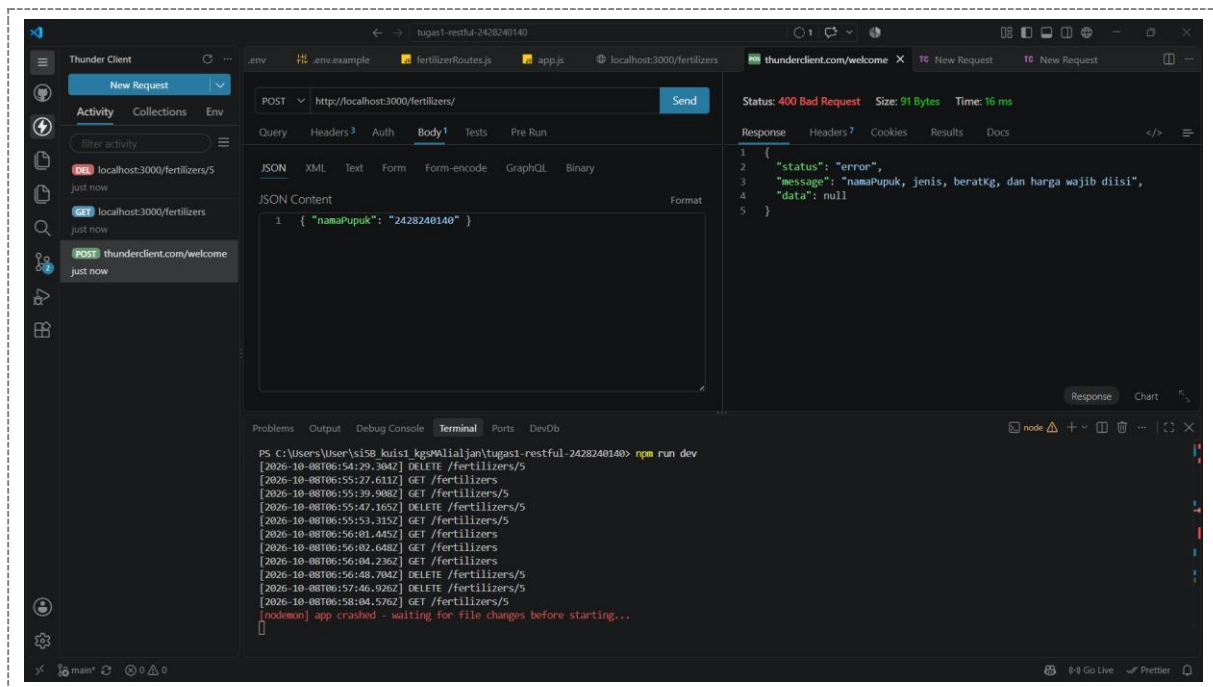
Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

Middleware cekApiKey yang menolak, karena header x-api-key tidak ada sehingga tidak sama dengan API_KEY. Ia langsung membalas 401 dengan return tanpa memanggil next(), sehingga createFertilizer tidak pernah dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /fertilizers
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPupuk": "2428240140" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — "namaPupuk, jenis, beratKg, dan harga wajib diisi"
Lulus?	☐ Ya ☐ Tidak



Gambar 10. POST dengan body tidak lengkap

Penjelasan:

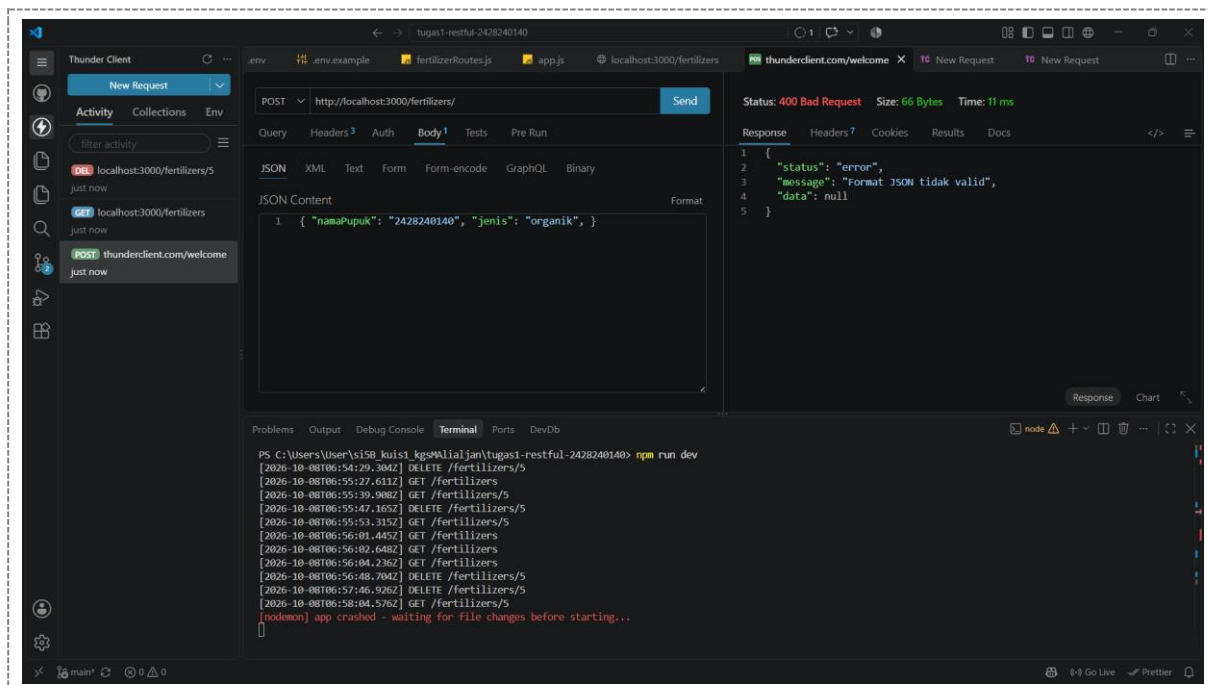
Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

Validasi ditulis di controller, pada fungsi validasi() yang dipanggil createFertilizer sebelum model dipakai. Statusnya 400 karena kesalahan ada pada data yang dikirim klien (field wajib kosong), bukan pada server.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /fertilizers
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPupuk": "2428240140", "jenis": "organik", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request — "Format JSON tidak valid"
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Penjelasan:

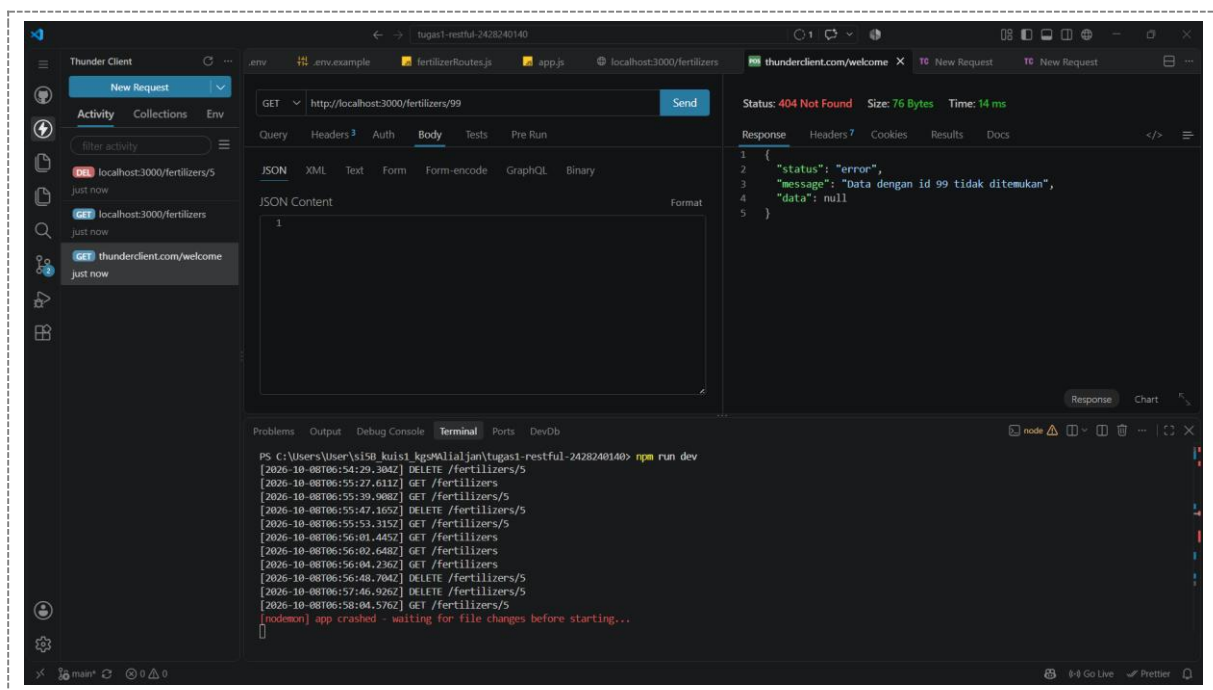
Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

`express.json()` yang mendeteksi JSON rusak dan melempar error bertipe `entity.parse.failed` sebelum controller dijalankan. `errorHandler` di `errorHandler.js` menangkap error itu dan mengubahnya menjadi respons 400 berformat JSON, sehingga bukan halaman HTML berisi jejak error.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /fertilizers/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — "Data dengan id 99 tidak ditemukan"
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Route router.get("/:id") memanggil getFertilizerById, yang memanggil Fertilizer.getById(99) dan mendapat undefined. Controller lalu memanggil fungsi tidakDitemukan() yang membuat pesan 404 dan mengirimkannya lewat res.status(404).json().

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	
T02	Menampilkan satu data berdasarkan ID	200	200	
T03	Menambah data baru	201	201	
T04	Mengubah data	200	200	
T05	Menghapus data	200	200	
N01	Menambah data tanpa API key	401	401	
N02	Menambah data dengan body tidak lengkap	400	400	
N03	Mengirim JSON yang rusak	400	400	
N04	Mengambil ID yang tidak ada	404	404	

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/alialjan/si5B_kuis1_alialjan
Pesan commit terakhir	Nambah id 1
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Tulis jawaban Anda di sini.

Route hanya memetakan alamat ke fungsi, misalnya `router.post("/", cekApiKey, controller.createFertilizer)` di `fertilizerRoutes.js`. Controller memeriksa input dan membuat response, misalnya `const pesan = validasi(body)` di `createFertilizer`. Model hanya mengolah data, misalnya `fertilizers.push(baru)` di fungsi `create()`, tanpa mengenal `req` dan `res`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Tulis jawaban Anda di sini.

Dengan error handler terpusat, format error cukup ditulis sekali di `errorHandler.js` dan semua rute otomatis memakainya, tidak perlu mengulang `res.status(...)` di setiap route. Buktinya pada N03: JSON rusak tidak ditangani di controller mana pun, tetapi tetap dibalas 400 berformat JSON "Format JSON tidak valid", dan alamat yang tidak terdaftar dibalas 404 JSON oleh `notFound`.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100